

Influence de la granularité des microservices et de leur déploiement conteneurisé sur les performances d'une application distribuée simulée.

Rapport d'application à la démarche scientifique

Résumé : La granularité des microservices et le mode de déploiement conteneurisé sont deux paramètres architecturaux majeurs dont l'impact combiné sur les performances reste peu étudié en simulation contrôlée. Cette étude évalue six configurations issues du croisement de trois niveaux de granularité et deux modes d'isolation conteneurisée, testées sur dix applications distribuées générées aléatoirement sous sept niveaux de charge. Les résultats montrent que l'isolation conteneurisée est le facteur déterminant à forte charge, réduisant la latence et les échecs au prix d'une consommation CPU élevée, et révèlent un découplage inattendu entre utilisation CPU et latence selon le mode de déploiement.

Mots-clés : microservices, granularité, performance, déploiement conteneurisé, application distribuée

Abstract: Microservice granularity and containerized deployment are two key architectural parameters whose combined impact on performance remains understudied in controlled simulation settings. This study evaluates six configurations derived from three granularity levels and two isolation modes, tested across ten randomly generated distributed applications under seven load levels. Results show that containerized isolation is the dominant factor under high load, reducing latency and failure rates at the cost of higher CPU consumption, and reveal an unexpected decoupling between CPU utilization and response time depending on the deployment mode.

Keywords: microservices, granularity, performance, containerized deployment, distributed application

Alban GODIER

CESI – École d'ingénieur – FISE A4 Science du Numérique

6 mai 2026

Table des matières

Table des figures	3
Introduction.....	4
I. Analyse documentaire	5
II. Méthodologie	7
1. Conception expérimentale.....	7
2. Environnement de simulation	7
3. Scénarios et variables	8
4. Mesures et métriques.....	10
5. Procédure d'exécution et collecte.....	10
a. Phase de préparation	11
b. Phase d'exécution	12
c. Phase d'agrégation et d'analyse des données.....	14
III. Résultats	15
1. Résultats principaux.....	15
a. Utilisation CPU	15
b. Durée de réponse.....	16
c. Taux d'échec	17
2. Analyse statistique	17
3. Résultats secondaires.....	18
a. Inversion du taux d'échec à 1 000 requêtes.....	18
b. Découplage entre consommation CPU et latence.....	18
c. Stabilité du monolithique à charge modérée	18
IV. Discussion	19
1. Interprétation des résultats.....	19
2. Comparaison avec la littérature.....	19
3. Implications pratiques	20
V. Limites et recherches futures.....	21
1. Limites méthodologiques.....	21
2. Validité et généralisabilité.....	21
3. Pistes de recherches futures.....	21
Conclusion.....	22
1. Synthèse des principaux enseignements	22
2. Recommandations	22
3. Perspectives	22
Glossaire	23
Bibliographie.....	26

Table des figures

Figure 1 : Architecture d'illustration pour une granularité fine et une isolation faible	8
Figure 2 : Architecture d'illustration pour une granularité moyenne et une isolation faible	8
Figure 3 : Architecture d'illustration pour une granularité grossière et une isolation faible	9
Figure 4 : Architecture d'illustration pour une granularité moyenne et une isolation moyenne	9
Figure 5 : Architecture d'illustration pour une granularité grossière et une isolation moyenne	9
Figure 6 : Architecture d'illustration pour une granularité grossière et une isolation forte	10
Figure 7 : Exemple de déroulement d'une requête de test dans le cas d'une architecture de 4 microservices, de granularité moyenne et d'isolation moyenne.....	13
Figure 8 : Moyenne d'utilisation du CPU en fonction de la charge et de la configuration.....	16
Figure 9 : Temps de réponse moyen en fonction de la charge et de la configuration	16
Figure 10 : Taux d'échec en fonction de la charge et de la configuration	17

Introduction

Lors de la conception d'une architecture d'une application, il est souvent nécessaire de faire le choix entre une architecture monolithique ou une architecture en microservices. Cette dernière est largement adoptée pour concevoir des systèmes distribués évolutifs et maintenables. Elle s'appuie sur le principe de découpage d'une application en services autonomes, chacun implémentant une fonctionnalité spécifique et communiquant avec les autres via des interfaces bien définies. On nomme ces composants logiciels des microservices et leur niveau de granularité, c'est-à-dire la taille et la portée fonctionnelle de chaque service, constitue une décision importante qui influence directement les performances du système. Une granularité fine favorise l'isolation des pannes et la scalabilité, mais augmente les charge réseau et la complexité. À l'inverse, une granularité plus grossière limite les communications réseau tout en réduisant l'indépendance des composants[1]. Cette étude vise à analyser expérimentalement, au travers de simulations, l'impact de différents niveaux de granularité sur les performances, afin d'identifier les compromis architecturaux les plus pertinents dans un contexte contrôlé.

Le choix de la simulation n'est pas anodin, car il permet de contrôler précisément les variables d'intérêt (granularité, charge de travail, etc.) et d'obtenir des données de performance dans un environnement isolé. Cependant, il est important de noter que les résultats obtenus à partir de simulations peuvent ne pas être entièrement représentatifs des conditions réelles d'une application distribuée en production. Les simulations peuvent simplifier certains aspects du système, tels que les interactions complexes entre les microservices, les variations de charge et les comportements imprévus. Par conséquent, les conclusions tirées de cette étude doivent être interprétées avec prudence et complétées par des études empiriques sur des applications réelles pour valider les résultats.

Les simulations ont été réalisées en utilisant des outils de conteneurisation tels que Docker, qui permettent de créer des environnements isolés pour chaque microservice. Nous avons utilisé des scripts Python pour simuler les microservices et générer des charges de travail variées. Les performances ont également été mesurées à l'aide de scripts Python permettant un monitoring en temps réel des ressources utilisées par chaque microservice.

I. Analyse documentaire

La granularité des microservices est un sujet largement documenté dans la littérature, avec des études qui mettent en évidence les avantages et les inconvénients de différentes approches. Par exemple, certains auteurs soutiennent que une granularité fine permet une meilleure isolation des pannes et une scalabilité accrue, tandis que d'autres soulignent que cela peut entraîner une complexité accrue et des coûts de communication plus élevés[2]. D'autres études ont également exploré les métriques utilisées pour évaluer la granularité des microservices, telles que le couplage, la cohésion et la complexité du code[3]. Enfin, des recherches récentes ont examiné l'impact de la granularité sur la consommation d'énergie et les performances, mettant en évidence des compromis importants à considérer lors de la conception d'une architecture en microservices[4].

Afin de constituer cette analyse documentaire, cinq articles ont été sélectionnés pour leur proximité avec la thématique étudiée après une recherche bibliographique approfondie partant de Google Scholar avec les termes clés "microservices AND granularity". Leur analyse a été structurée grâce à la méthode QALMRI qui permet d'extraire les éléments clés de chaque étude, tels que les questions de recherche, les hypothèses, les méthodes utilisées, les résultats obtenus et les conclusions.

Les résultats de cette analyse documentaire ont permis de sélectionner un article de référence couvrant principalement la définition de la granularité des microservices et les métriques associées[3], ainsi que quatre articles traitant empiriquement l'impact de la granularité sur les performances[1], [2], [4], [5].

La première tendance identifiée dans la littérature est le coût de la communication entraîné par une granularité fine. En effet, plus les microservices sont petits et nombreux, plus ils doivent communiquer entre eux pour réaliser leurs tâches, ce qui peut entraîner une augmentation de la latence et une diminution des performances globales du système[2]. Toutefois, il faut noter que cette augmentation de la latence est faible dans les systèmes de petite taille, mais devient significative à mesure que la taille du système augmente[4].

Cette tendance est confirmée par une étude plus récente qui a évalué l'impact de la granularité sur les performances d'une application distribuée[5]. Les résultats ont montré que le découpage en microservices peut entraîner une augmentation significative de la latence, en particulier lorsque les microservices sont déployés sur des machines virtuelles distinctes. Cette tendance semble confirmée par l'expérience menée par Camila Medeiros Rêgo, Ricardo César Mendonça Filho et Nabor C. Mendonça[1], qui a montré que la distribution des services sur plusieurs nœuds peut améliorer la scalabilité, mais au prix d'une charge de communication plus importante.

Un deuxième axe semble se dessiner en ce qui concerne la consommation des ressources du système dans son ensemble. En effet, une granularité fine peut entraîner une utilisation plus importante des ressources, notamment en termes de mémoire et de CPU, en raison de la multiplication des processus et des communications entre les microservices[4]. Cependant, il est important de noter que cette augmentation de l'utilisation des ressources peut être compensée par les avantages en termes de scalabilité et d'isolation des pannes offerts par une granularité fine.

L'ensemble de ces sources semble converger vers une amélioration de la modularité, de l'isolement et de la scalabilité en adoptant une granularité fine, mais au prix d'une augmentation de la latence et de l'utilisation des ressources. Il est donc crucial de trouver un équilibre entre ces différents facteurs pour concevoir une architecture en microservices efficace et performante. Il est également important de noter que les résultats obtenus dans ces études montrent que la relation entre granularité et performance n'est pas linéaire car elle dépend de l'architecture utilisée, de la charge de travail, du contexte d'exécution ou encore de la méthode de communication inter-service choisie.

Pour conclure cette analyse, il semble y avoir deux limites récurrentes dans les études traitant de l'impact de la granularité sur les performances. Tout d'abord, la plupart des études se concentrent

sur des systèmes de petite taille, ce qui limite la généralisation des résultats à des systèmes plus complexes et de grande taille. Ensuite, peu de travaux se concentre explicitement sur la granularité, le déploiement conteneurisé contrôlé et les performances dans un cadre de simulation. Cette étude vise à combler ces lacunes en évaluant ces différentes métriques dans un environnement simulé et maîtrisé afin d'évaluer comment plusieurs niveaux de granularité et de déploiement conteneurisé influencent les performances d'une application distribuée.

II. Méthodologie

1. Conception expérimentale

L'objectif de cette étude est d'évaluer l'impact de la granularité des microservices et de leur déploiement conteneurisé sur les performances d'une application distribuée simulée. Pour ce faire, nous avons défini trois constituants d'un système distribué simulé :

- Le microservice, représentant une tâche simple
- Le service, regroupant un ensemble de microservices et représentant une fonctionnalité plus large accessible via une API
- Le conteneur, regroupant un ensemble de services et symbolisant l'isolement matériel et logiciel.

Nous avons ensuite conçu une expérience contrôlée articulée autour de trois niveaux de granularité

- Fin : chaque microservice s'exécute dans un service dédié
- Moyen : plusieurs microservices (entre 1 et 4) regroupés dans un même service
- Grossier : tous les microservices regroupés dans un seul service

Nous avons également trois niveaux de conteneurisation :

- Un conteneur unique regroupant tous les services (application monolithique)
- Plusieurs conteneurs regroupant entre 1 et 4 services
- Un conteneur par service (isolement maximal)

Ces deux facteurs nous ont permis de formuler les hypothèses suivantes :

- (H1) une granularité plus fine entraînera une augmentation de la latence en raison de la multiplication des communications interservices
- (H2) le déploiement multi-conteneur augmentera la latence en raison de la surcharge liée à la virtualisation
- (H3) à l'inverse, une granularité grossière entraînera une dégradation des performances sous forte charge, en raison de la concentration des requêtes sur un seul service.

2. Environnement de simulation

L'environnement de simulation est basé sur Docker (version 20.10.0) et Docker Compose (version 2.1.2). Chaque service est simulé par un script Python (version 3.10.4, en environnement virtuel) démarrant un serveur HTTP simple multi-threadé, ce qui permet d'éviter les blocages liés à une file d'attente trop courte et de se rapprocher d'un fonctionnement réaliste. Les microservices effectuent une tâche de calcul représentative d'une charge de travail réelle : 20 000 itérations d'une division euclidienne. Les conteneurs sont configurés avec une limite d'utilisation CPU afin de simuler les contraintes d'un système distribué réel et d'accentuer les différences de performances entre configurations. L'ensemble de la simulation est orchestré via Docker Compose, permettant de déployer rapidement les différentes configurations et de collecter les métriques de performance à l'aide de scripts Python dédiés au monitoring en temps réel. Les simulations ont été exécutées sur un ordinateur personnel sous Windows 11.

3. Scénarios et variables

Les constituants définis ci-dessus permettent de définir les six configurations suivantes (nommées de 1 à 6 selon l'ordre croissant de granularité et d'isolation) :

1. Granularité fine + isolation faible (Figure 1)

La figure ci-dessous (comme les suivantes) ne représente pas une architecture réellement utilisée dans la simulation mais un exemple de se qu'elle pourrait être. Elle sert avant tout à illustrer le découpage en services et conteneurs.

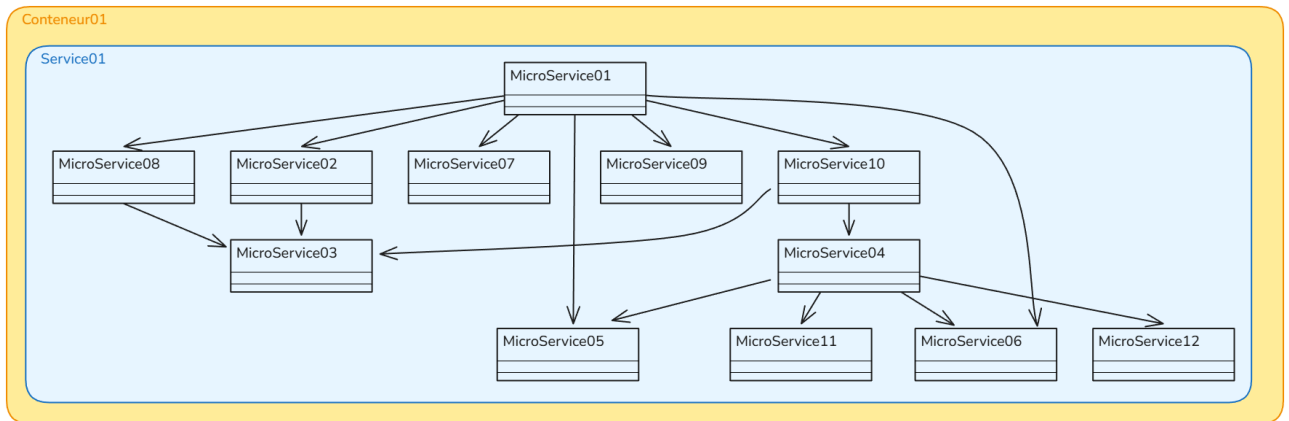


Figure 1 : Architecture d'illustration pour une granularité fine et une isolation faible

2. Granularité moyenne + isolation faible (Figure 2)

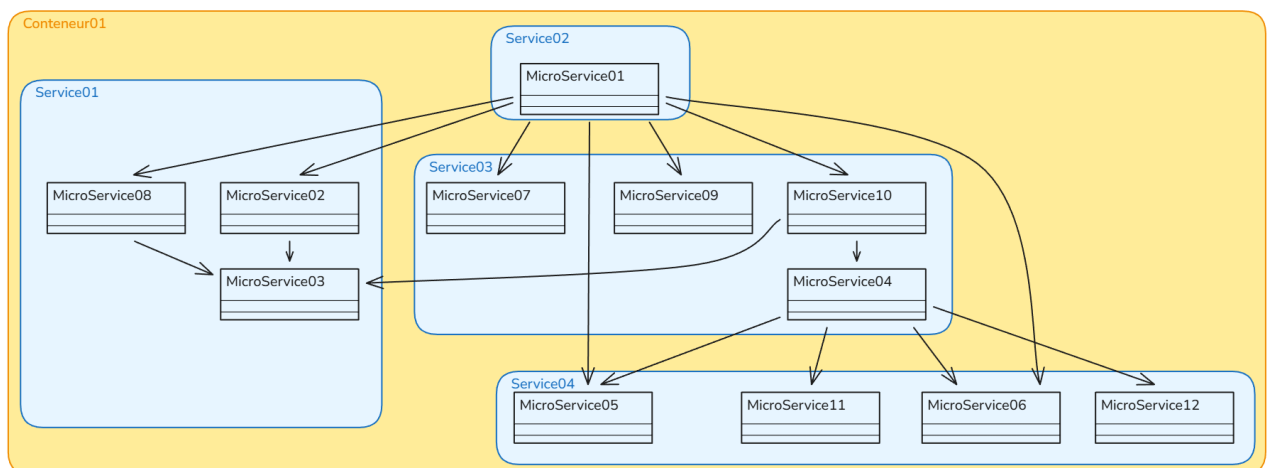


Figure 2 : Architecture d'illustration pour une granularité moyenne et une isolation faible

3. Granularité grossière + isolation faible (Figure 3)

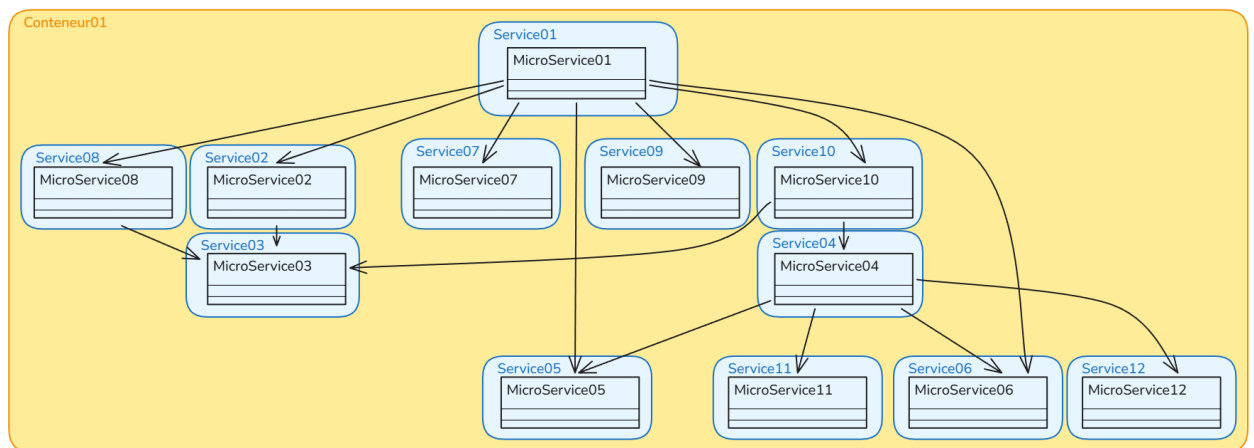


Figure 3 : Architecture d'illustration pour une granularité grossière et une isolation faible

4. Granularité moyenne + isolation moyenne (Figure 4)

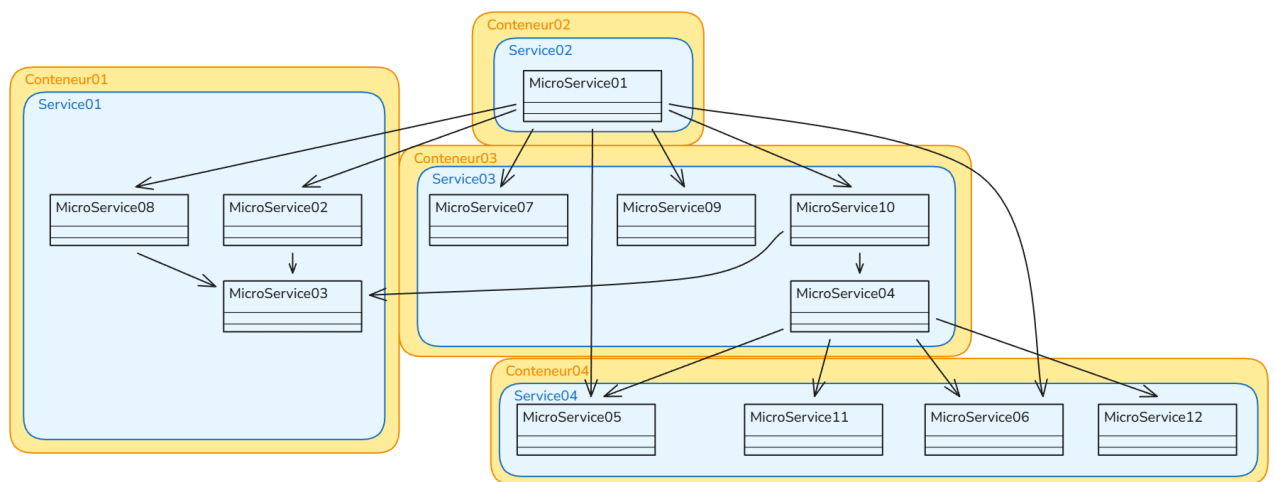


Figure 4 : Architecture d'illustration pour une granularité moyenne et une isolation moyenne

5. Granularité grossière + isolation moyenne (Figure 5)

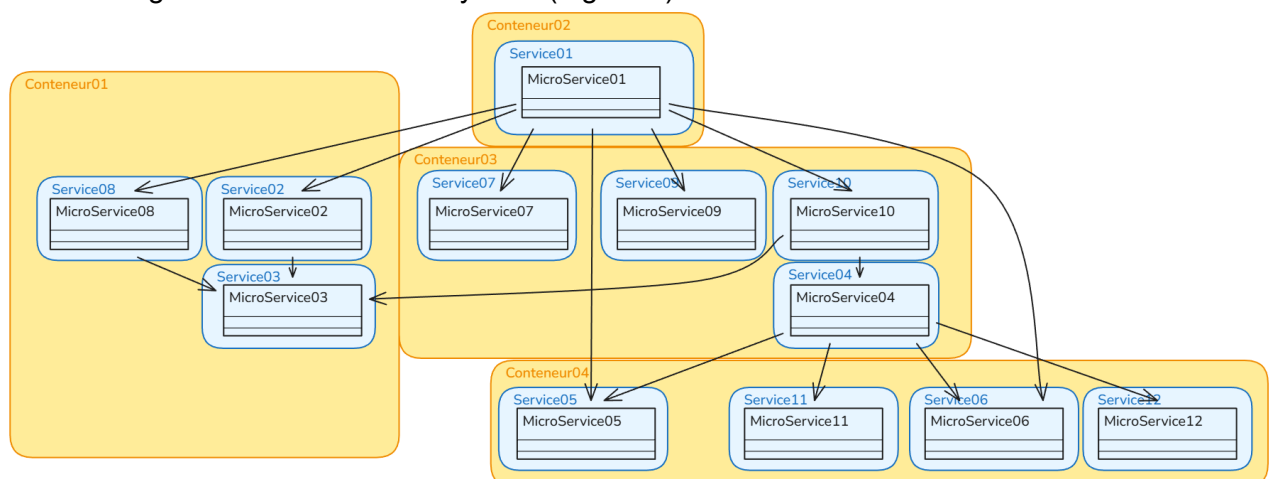


Figure 5 : Architecture d'illustration pour une granularité grossière et une isolation moyenne

6. Granularité grossière + isolation forte (Figure 6)

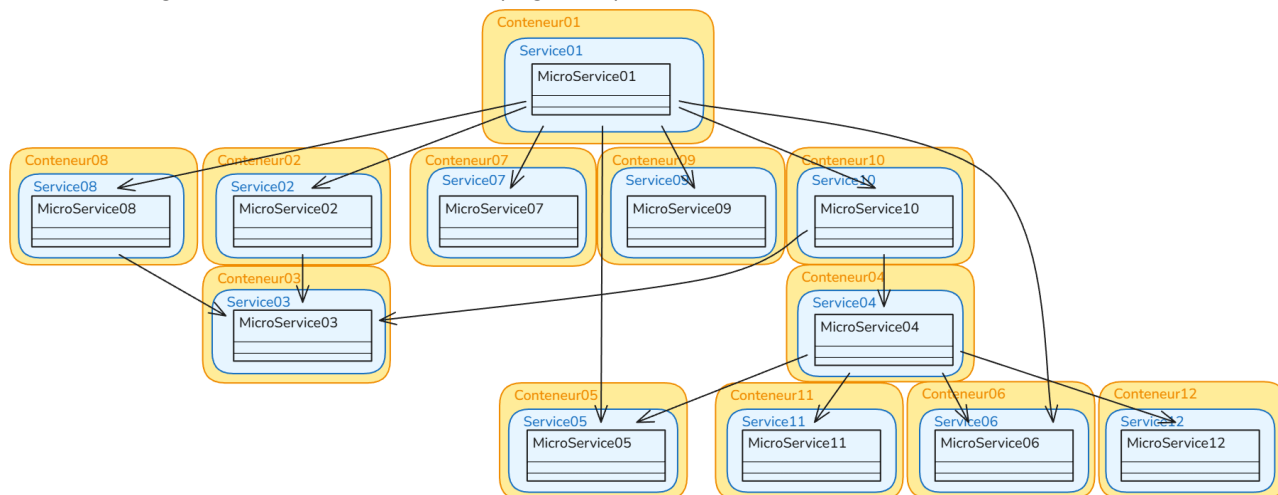


Figure 6 : Architecture d'illustration pour une granularité grossière et une isolation forte

Comme on peut le constater, ces configurations ne couvrent pas toutes les combinaisons possibles entre les trois niveaux de granularité et d'isolation. Les configurations non représentées (par exemple, granularité fine + isolation forte) ont été volontairement exclues car elles ne sont pas possibles. Par exemple, dans le cas d'une granularité fine et une isolation forte, il n'est pas possible de répartir des microservices d'un même service dans des conteneurs distincts.

4. Mesures et métriques

Trois métriques principales sont collectées pour chaque expérience :

- La latence moyenne (en millisecondes) correspondant au temps de réponse moyen des requêtes HTTP émises vers les services.
- Le nombre d'échecs de requêtes (en pourcentage) correspondant au taux de requêtes ayant échoué (par exemple, en raison d'un timeout ou d'une erreur de serveur).
- L'utilisation CPU moyenne par conteneur (en pourcentage).

Les ressources des conteneurs sont mesurées via la commande *docker stats*. Les temps de réponse et les échecs sont enregistrés par les scripts Python qui émettent des requêtes HTTP vers les services et consignent les réponses.

Les données sont enregistrées sous la forme de fichiers CSV, puis compilées pour chaque configuration et niveau de charge sous la forme d'un histogramme représentant les trois métriques en fonction de la configuration et du niveau de charge.

5. Procédure d'exécution et collecte

Les expériences se déroule en trois phases :

1. Phase de préparation :
2. Phase d'exécution
3. Phase d'agrégation et d'analyse des données

a. Phase de préparation

Cette étape consiste à la génération de 10 applications distribuées aléatoires, c'est à dire de 12 microservices dont les dépendances sont générées aléatoirement. Une dépendance est une relation de type "le microservice A doit appeler le microservice B pour réaliser sa tâche". Le premier microservice de chaque application est considéré comme le point d'entrée de l'application, c'est à dire que c'est lui qui reçoit les requêtes HTTP émises par les scripts Python de test, il ne peut donc pas être une dépendance d'un autre microservice. Ensuite, chaque application est déclinée selon les 6 configurations définies précédemment, produisant 60 configurations au total.

Le programme python responsable de cette phase de préparation est exécuté en utilisant la commande :

```
python -m src generate-configs --output ./output --count 10
```

À la fin de cette phase, nous obtenons un dossier contenant :

```
.output/
├── 001
│   ├── 1_monolithic
│   │   ├── config.json
│   │   ├── docker-compose.yml
│   │   ├── Dockerfile
│   │   ├── README.md
│   │   ├── requirements.txt
│   │   ├── runtime_runner.py
│   │   └── src/ ...
│   ├── 2_microservices_medium_granularity/ ...
│   ├── 3_microservices_fine_granularity/ ...
│   ├── 4_microservices_medium_granularity_isolation/ ...
│   ├── 5_microservices_fine_granularity_isolation/ ...
│   └── 6_microservices_fine_granularity_high_isolation / ...
├── ...
└── 010 / ...
```

Où :

- 001 à 010 représentent les 10 applications distribuées générées aléatoirement
- 1 à 6 représentent les 6 configurations définies précédemment
- config.json contient la configuration de l'application (dépendances entre microservices, etc.)
- docker-compose.yml contient la configuration de Docker Compose pour le déploiement de l'application
- Dockerfile contient la configuration de Docker pour la construction de l'image de l'application
- README.md contient un graphe représentant les dépendances entre les microservices de l'application et les services et conteneurs dans lesquels ils sont regroupés
- requirements.txt contient les dépendances Python nécessaires à l'exécution de l'application
- runtime_runner.py contient le code nécessaire pour l'exécution des serveurs HTTP des services
- src/ contient le code source des services et microservices de l'application

Une fois cette phase de préparation terminée, nous sommes prêts à passer à la phase d'exécution des expériences. En l'état actuel, chaque sous-dossier (de 1 à 6) contient une configuration de

l'application pouvant être exécutée via Docker Compose totalement indépendamment des autres configurations.

Ces fichiers sont disponibles dans le dossier du projet fourni avec le rapport ou sur le [Github](#).

b. Phase d'exécution

Cette étape consiste à exécuter chaque configuration de chaque application sous différentes charges de travail, et à collecter les métriques de performance (latence, échecs, utilisation CPU) pour chaque expérience. Les expériences sont exécutées en utilisant la commande suivante :

```
python -m src test-all --output ./output --requests  
"1;5;10;50;100;500;1000;5000" --missing
```

Cette commande permet de tester toutes les configurations de toutes les applications générées précédemment, et de collecter les métriques de performance pour chaque expérience. Les résultats sont stockés dans des fichiers CSV structurés par niveau de charge. Ainsi, une fois toutes les expériences exécutées, le dossier d'une configuration d'une application contient alors :

```
1_monolithic  
├── ...  
├── result_1.csv  
├── result_5.csv  
├── result_10.csv  
├── result_50.csv  
├── result_100.csv  
├── result_500.csv  
├── result_1000.csv  
└── result_5000.csv
```

Où result_X.csv contient les métriques de performance (latence, échecs, utilisation CPU) pour la configuration testée sous une charge de travail de X requêtes simultanées.

Une requête typique envoyée par les scripts de test parcourt les microservices de la façon suivante dans le cas d'une architecture de 4 microservices, de granularité moyenne et d'isolation moyenne (cf Figure 7) :

1. Le script de test fait une requête au point d'entrée de l'architecture, le microservice 01.
2. Le microservice 01 a pour dépendance le microservice 02, il lui fait donc une requête. Celui-ci est en dehors du même service que le microservice 01, il doit donc faire une requête HTTP pour communiquer avec lui.
3. Le microservice 02 a pour dépendance le microservice 04 et ils sont tous les deux regroupés dans le même service, il peut donc communiquer avec lui via un simple appel de fonction comme si les deux tâches avaient été développées dans un seul programme.
4. Le microservice 04 ne dépend d'aucun autre microservice, il effectue donc sa tâche de calcul et retourne une réponse au microservice 02.
5. Le microservice 02 a une autre dépendance, le microservice 03, il lui fait donc une requête HTTP pour communiquer avec lui car ils ne sont pas dans le même service.
6. Le microservice 03 ne dépend d'aucun autre microservice, il effectue donc sa tâche de calcul et retourne une réponse au microservice 02.
7. Le microservice 02 retourne une réponse au microservice 01.
8. Le microservice 01 retourne une réponse au script de test.

Ces fichiers sont disponibles dans le dossier du projet fourni avec le rapport ou sur le [Github](#).

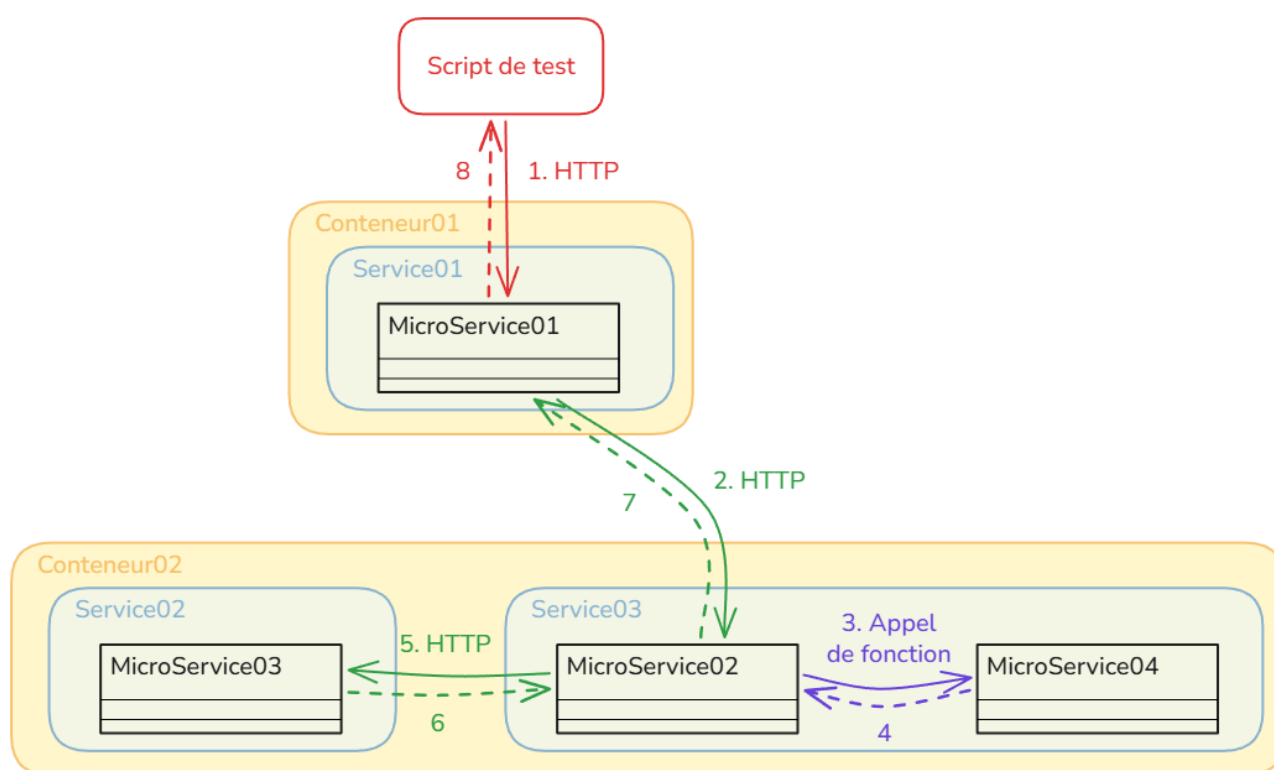


Figure 7 : Exemple de déroulement d'une requête de test dans le cas d'une architecture de 4 microservices, de granularité moyenne et d'isolation moyenne

Après une dizaine d'heures d'exécution, nous obtenons ainsi 480 fichiers CSV contenant les résultats de chaque expérience. On peut maintenant passer à la phase d'agrégation et d'analyse des données.

c. Phase d'agrégation et d'analyse des données

Cette étape consiste à agréger les données collectées lors de la phase d'exécution, afin de faciliter l'analyse et la comparaison des performances entre les différentes configurations. L'agrégation consiste à calculer les moyennes et les écarts-types pour chaque configuration et niveau de charge. Nous avons alors pu tracer trois histogrammes représentant respectivement les trois métriques en fonction de la configuration et du niveau de charge. L'ensemble de ces étapes est réalisé à l'aide de la commande :

```
python -m src plot-results --plot-type bar --output ../output/plots
```

III. Résultats

Cette partie se concentre sur l'analyse des histogrammes ci-contre (Figure 8, Figure 9 et Figure 10) qui présentent respectivement la latence moyenne, le taux d'échec et l'utilisation CPU moyenne en fonction de la configuration et du niveau de charge.

Les données des graphiques sont issues de l'agrégation des 480 fichiers CSV sous la forme de 3 fichiers CSV :

- `failure_rate_percent_table.csv` : contient le taux moyen d'échec et l'erreur standard correspondante pour chaque niveau de charge et chaque configuration.
- `mean_cpu_usage_percent_table.csv` : contient le taux moyen d'utilisation du processeur et l'erreur standard correspondante pour chaque niveau de charge et chaque configuration.
- `mean_response_duration_ms_table.csv` : contient la durée moyenne de réponse et l'erreur standard correspondante pour chaque niveau de charge et chaque configuration.

Ces fichiers sont disponibles dans le dossier du projet fourni avec le rapport ou sur le [Github](#).

1. Résultats principaux

Les résultats obtenus sur les 6 configurations testées montrent des tendances claires sur les trois métriques : ressources moyenne CPU consommées, durée moyenne d'une requête et taux d'échec moyen. Les configurations se comportent de façon similaire face à une faible charge (entre 1 et 10 requêtes). Des différences commencent à se voir à partir de 50 requêtes simultanées et s'accroissent beaucoup à partir de 500 requêtes.

a. Utilisation CPU

À faible charge, toutes les configurations montrent une utilisation CPU proche de 0, sans différence significative. Une divergence apparaît ensuite, à partir de 50 requêtes : les configurations sans isolation (1 à 3) consomment entre 25% et 50% de CPU, alors que les configurations avec isolation (4 à 6) consomment entre 85% et 110% de CPU. Cette tendance s'accroît après 500 requêtes, où la configuration 6 (granularité fine, isolation maximale) dépasse 210% d'utilisation CPU en moyenne, avec des intervalles de confiance très larges traduisant une forte instabilité. Les configurations sans isolation se stabilisent autour de 45 à 50% quelle que soit la charge, cela montre une saturation de l'unique conteneur utilisé pour le déploiement. (Figure 8)

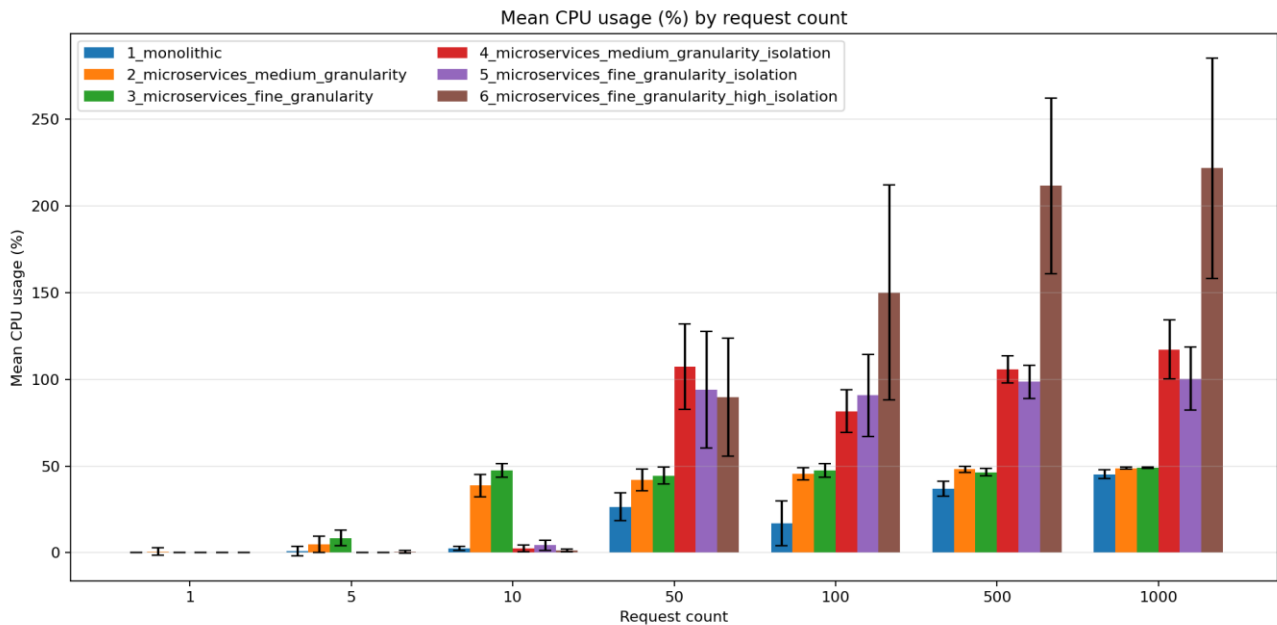


Figure 8 : Moyenne d'utilisation du CPU en fonction de la charge et de la configuration

b. Durée de réponse

La durée de réponse moyenne ou latence moyenne reste sous 2 000 ms pour toutes les configurations jusqu'à 100 requêtes en simultanées. Au-delà, les configurations sans isolation (2 et 3) montrent une croissance rapide de la latence qui atteint respectivement environ 65 000 ms et 70 000 ms à 1 000 requêtes. La configuration monolithique (1) affiche une latence intermédiaire d'environ 12 500 ms. À l'inverse, les configurations avec isolation conteneurisée (4 à 6) maintiennent des durées de réponse significativement inférieures à charge élevée. Ces résultats indiquent que l'isolation conteneurisée, bien qu'elle augmente la consommation CPU, permet de réduire efficacement la latence sous forte charge. (Figure 9)

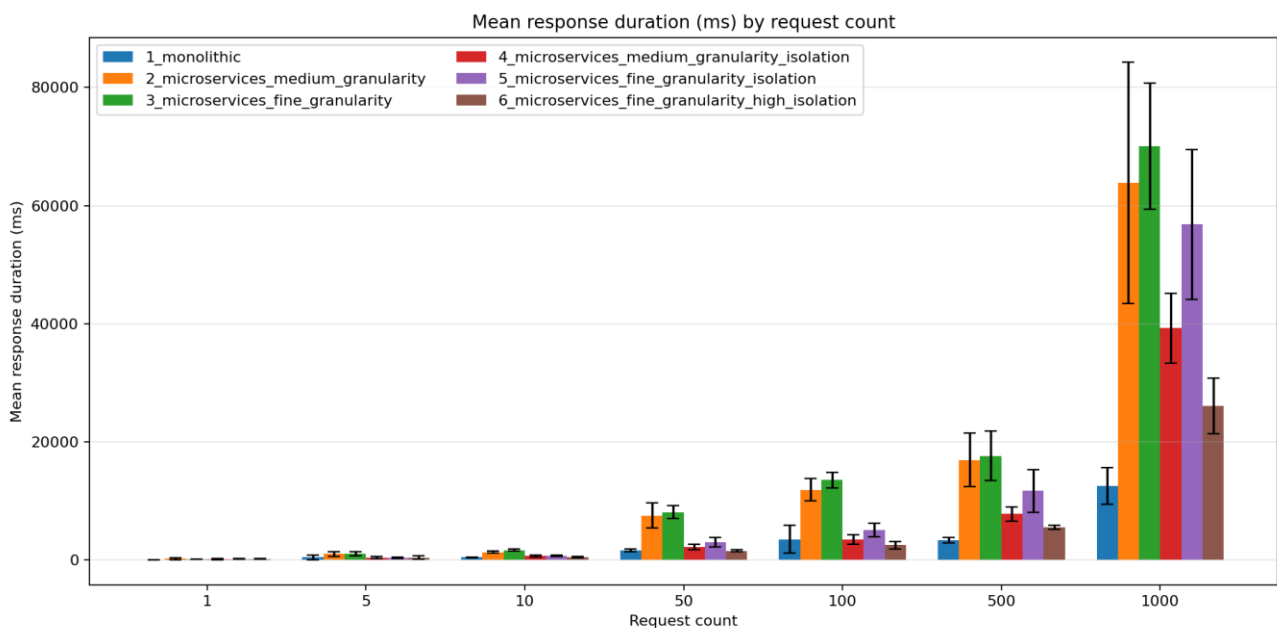


Figure 9 : Temps de réponse moyen en fonction de la charge et de la configuration

c. Taux d'échec

On ne constate aucun échec pour les niveaux de charge allant de 1 à 10 requêtes. Les premiers apparaissent à 50 requêtes à des taux plutôt faible compris entre 4% et 8% si on considère que le service doit gérer au minimum 50 requêtes en même temps sur un seul serveur. À 500 requêtes, le taux d'échec atteint son maximum avec la configuration monolithique (1) qui culmine à environ 68%, suivie des autres entre 40 et 60% d'échec. À 1 000 requêtes, une inversion étrange de la tendance se distingue nettement avec une diminution du taux d'échec pour toutes les configurations. Ce comportement contre-intuitif s'explique par la saturation du système dès 500 requêtes, qui entraîne un rejet dès le début des requêtes en surcharge. On notera, cependant, que les configurations avec isolation ont tendance à mieux gérer la saturation et à limiter les échecs, avec la configuration 6 qui atteint un taux d'échec quasi nul à 1 000 requêtes (2%). (Figure 10)

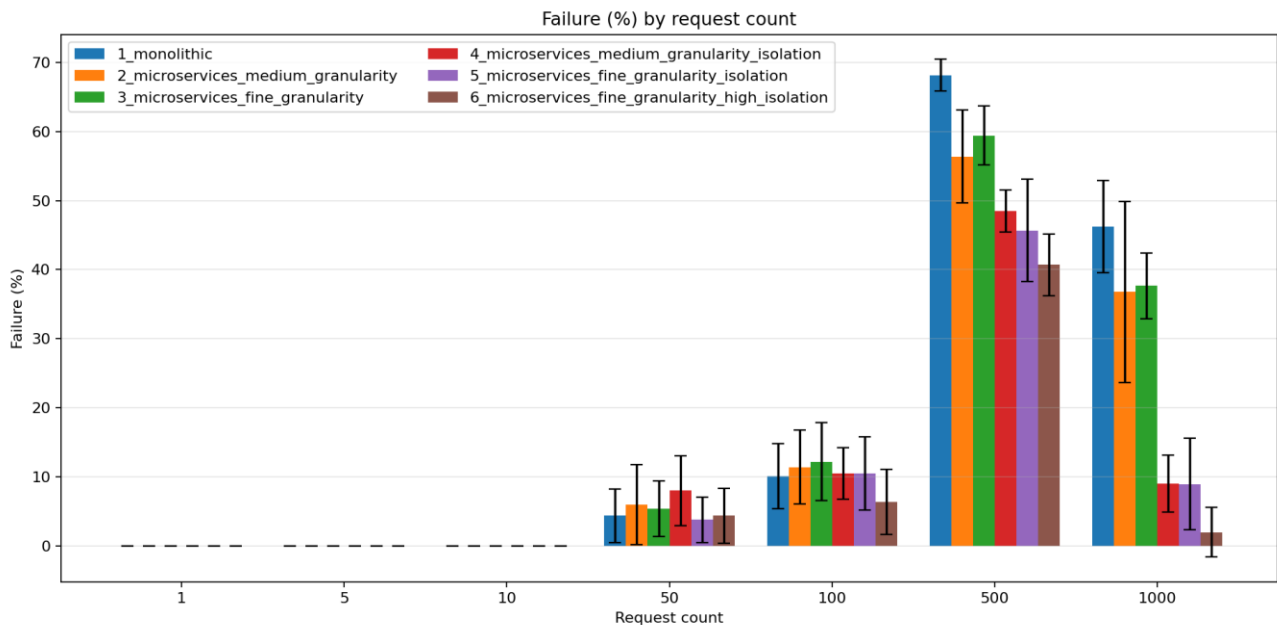


Figure 10 : Taux d'échec en fonction de la charge et de la configuration

2. Analyse statistique

Les barres d'erreur représentées sur chaque graphique correspondent aux intervalles de confiance à 95% calculés sur les dix applications distribuées générées aléatoirement pour chaque configuration. Leur amplitude varie significativement selon la métrique et le niveau de charge considérés.

Pour l'utilisation CPU, les intervalles de confiance sont étroits à faible charge mais s'élargissent fortement à partir de 100 requêtes pour la configuration 6, indiquant une instabilité importante. Cela suggère que l'architecture de l'application devient un facteur important dans la consommation de ressources.

Pour la durée de réponse, les intervalles de confiance restent relativement faibles jusqu'à 1 000 requêtes. Cela confirme l'instabilité relevée ci-dessus. On peut toutefois noter que celle-ci est moins représentée dans le cas d'une architecture avec isolation.

3. Résultats secondaires

a. Inversion du taux d'échec à 1 000 requêtes

Le comportement le plus inattendu concerne la diminution du taux d'échec entre 500 et 1 000 requêtes pour toutes les configurations. Ce phénomène suggère qu'au-delà d'un seuil de saturation, les architectures rejettent activement les requêtes entrantes limitant ainsi le nombre de traitements tentés et donc d'échecs enregistrés. Ce mécanisme est très marqué pour la configuration 6, qui retrouve une quasi-absence d'échecs à 1 000 requêtes, au prix d'une consommation CPU très élevée.

b. Découplage entre consommation CPU et latence

Un des résultats les plus importants est l'absence de corrélation directe entre l'utilisation du CPU et la durée de réponse en fonction du mode de déploiement. Les configurations sans isolation 2 et 3 présentent des latences plus élevées à forte charge malgré une consommation CPU d'environ 45% tandis que la configuration 6 montre une latence plus faible à 1 000 requêtes malgré une utilisation du CPU dépassant 200 %. Cette divergence s'explique par le fait que la configuration avec une forte isolation répartie la charge sur plus de processus parallèle, augmentant la consommation globale mais réduisant les temps d'attente par requête.

c. Stabilité du monolithique à charge modérée

La configuration monolithique (1) se trouve être compétitive jusqu'à 100 requêtes simultanées, affichant des performances comparables aux configurations microservices en termes de latence et de taux d'échec, pour une consommation CPU inférieure. Ce n'est qu'à partir de 500 requêtes que sa limitation architecturale apparaît avec un taux d'échec plus élevé que les autres configurations (68%). Cette observation confirme l'hypothèse H3 selon laquelle la concentration du traitement dans un service unique crée des surcharges lors d'une forte demande.

IV. Discussion

1. Interprétation des résultats

Les résultats mettent en évidence trois mécanismes principaux.

À faible charge (1 à 10 requêtes), l'architecture n'a pas d'influence mesurable sur les performances : toutes les configurations se comportent de manière similaire, cela suggère que l'augmentation de la communication inter-services est négligeable lorsque l'application est peu sollicitée.

À charge moyenne (50 à 100 requêtes), deux logiques s'opposent. Dans la première, les configurations sans isolation (2 et 3) consomment peu de CPU grâce à des communications légères entre les services. Dans la seconde, la virtualisation a un coût important, mais le traitement en parallèle qu'elle apporte permet de garder la latence et le taux d'échec à des valeurs comparables au premier cas.

Enfin, à forte charge (500 à 1,000 requêtes), le facteur limitant bascule : ce n'est plus le surcoût en communication qui détermine les performances, mais la capacité du système à gérer les requêtes concurrentes. Les configurations sans isolation saturent rapidement car elles ne peuvent paralléliser le traitement sur plusieurs processus indépendants. Le monolithe (1) est caractéristique de ce phénomène avec le taux d'échec le plus élevé à 500 requêtes (68%), confirmant l'hypothèse H3. À l'inverse, la dernière configuration (6) tire parti de l'isolation maximale pour maintenir la latence la plus basse au prix d'une forte utilisation du CPU (plus de 200%).

2. Comparaison avec la littérature

Les résultats obtenus s'inscrivent globalement dans la continuité des travaux existants, tout en apportant des nuances.

La tendance observée (granularité fine sans isolation entraîne les latences les plus élevées à forte charge) est en accord avec les conclusions de Ricardo César Mendonça Filho et Nabor C. Mendonça[5], qui montrent que le découplage des services augmente significativement la surcharge de communication entre processus.

De plus, comme le souligne les observations de Yiming Zhao, Tiziano De Matteis et Justus Bogner[4], l'impact de la granularité augmente avec la taille de l'application, ce qui rejoint nos observations de l'instabilité des performances à mesure que la charge augmente.

À l'inverse, deux résultats s'éloignent des conclusions de la littérature. Le premier concerne l'impact de la conteneurisation sur la latence. Alors que Dharmendra Shadija, Mo Rezai et Richard Hill[2] concluent à une augmentation négligeable, notre étude nuance ce constat : si l'écart est faible à faible charge, il devient non négligeable à partir de 500 requêtes. Cela suggère que la visibilité du surcoût dépend de la charge du système.

Le second résultat concerne la séparation entre consommation CPU et latence. Contrairement à l'intuition selon laquelle une consommation CPU élevée traduit une baisse des performances, les configurations à isolation maximale combinent ici la plus forte utilisation CPU et la latence la plus faible à haute charge. Ce découplage n'a pas été abordé dans les études de référence et il constitue une conclusion non attendue.

3. Implications pratiques

Ces résultats permettent de formuler trois recommandations à destination des architectes logiciels.

Privilégier l'isolation conteneurisée dès que la charge anticipée dépasse un seuil critique. En dessous de 50 requêtes simultanées, le mode de déploiement importe peu. Au-delà, l'isolation devient un levier efficace pour contenir la latence et le taux d'échec, au prix d'une consommation CPU plus élevée.

Éviter l'architecture monolithique pour les systèmes à fort trafic. Malgré des performances compétitives à faible charge, le monolithe présente le taux d'échec le plus élevé à 500 requêtes. Sa simplicité ne compense pas son incapacité à répondre à toutes les requêtes sous forte charge.

Adapter la granularité à l'objectif prioritaire. Une granularité fine avec isolation maximale (configuration 6) offre la plus faible latence à haute charge, mais mobilise des ressources CPU importantes et présente une variabilité élevée selon la topologie de l'application (il faudra donc faire des tests par architecture). Une granularité moyenne avec isolation (configuration 4) offre un meilleur compromis, combinant une plus faible consommation CPU et des performances satisfaisantes.

V. Limites et recherches futures

1. Limites méthodologiques

Cette étude repose sur une simulation contrôlée qui simplifie certains aspects des systèmes distribués réels. Les microservices simulés implémentent des tâches génériques sans modéliser des comportements complexes tels que des accès concurrents à des ressources ou des appels à des services externes. De plus, nous n'avons pu générer que 10 architectures avec 12 microservices chacune, ce qui ne couvre pas l'espace complet des architectures rencontrées en production (cela a pris une dizaine d'heure pour toutes les tester). Enfin, l'étude ne se penche que sur 3 métriques alors que les facteurs importants en production peuvent être tout autre.

2. Validité et généralisabilité

La validité interne de l'expérience est assurée par le contrôle strict des variables : même matériel, mêmes scripts de génération de charge, mêmes conditions d'exécution pour toutes les configurations. Les résultats sont donc comparables entre eux au sein du protocole défini. En revanche, la validité externe est limitée : les conclusions s'appliquent majoritairement à des systèmes distribués de taille moyenne, dans un environnement sans contraintes réseau artificielles ni variabilité d'infrastructure. Leur comparaison à des environnements cloud multi-régions ou à des systèmes comportant des centaines de services doit être faite avec prudence.

3. Pistes de recherches futures

Trois axes se dégagent. En premier, une validation sur des systèmes réels permettrait de confronter les résultats de simulation aux comportements émergents des orchestrateurs de conteneurs (comme Kubernetes). En second lieu, l'intégration de la consommation énergétique comme métrique principale permettrait une analyse coût-performance plus complète, en lien avec les enjeux de sobriété numérique. Enfin, l'introduction d'injections de pannes contrôlées dans le protocole expérimental permettrait d'évaluer en même temps performance et résilience (capacité d'un système à se remettre d'une panne).

Conclusion

1. Synthèse des principaux enseignements

Cette étude a évalué l'influence de la granularité des microservices et du déploiement conteneurisé sur les performances d'une application distribuée simulée, à travers six configurations croisées testées sur dix applications et sept niveaux de charge. Les résultats montrent que le mode de déploiement est le facteur le plus important à forte charge : l'isolation conteneurisée permet de contenir la latence et le taux d'échec au prix d'une consommation CPU élevée, tandis que les architectures sans isolation saturent rapidement. Le monolithique, compétitif à charge modérée, présente le taux d'échec le plus élevé au-delà de 500 requêtes.

2. Recommandations

Pour les applications à charge faible ou prévisible, une architecture monolithique ou à granularité moyenne sans isolation reste pertinente et moins coûteuse en ressources. Pour les systèmes soumis à des pics de charge importants, une granularité moyenne avec isolation conteneurisée (configuration 4) constitue le meilleur compromis entre performance, prévisibilité et consommation CPU. La granularité fine avec isolation maximale (configuration 6) est à réserver aux systèmes dont la priorité absolue est la limitation des échecs, à condition de disposer des ressources matérielles nécessaires pour absorber la surcharge CPU induite.

3. Perspectives

Au-delà du cadre académique de cette étude, les compromis identifiés entre granularité, isolation et charge constituent des leviers concrets pour les équipes d'architecture logicielle confrontées à la conception d'une architecture logicielle. L'émergence d'outils comme Service Weaver permettant de modifier le niveau de granularité sans changer le code applicatif ouvre la voie à une optimisation dynamique de l'architecture en fonction des conditions d'exploitation observées. La question de la granularité optimale ne se pose alors plus uniquement au moment de la conception, mais tout au long du cycle de vie du système.

Glossaire

A

API — Interface de programmation applicative définissant un ensemble de règles permettant à deux composants logiciels de communiquer entre eux. Dans cette étude, chaque microservice expose une API HTTP que les scripts de génération de charge interrogent pour simuler les requêtes utilisateur.

Application distribuée — Système logiciel dont les composants s'exécutent sur plusieurs processus ou machines distinctes et communiquent entre eux via un réseau, afin de répartir la charge de traitement et d'améliorer la disponibilité du service.

C

Conteneur Environnement d'exécution isolé qui embarque un processus applicatif et ses dépendances, garantissant la reproductibilité de l'exécution indépendamment de l'infrastructure hôte. Docker est l'implémentation la plus répandue de ce concept.

CPU Unité centrale de traitement d'un ordinateur, responsable de l'exécution des instructions. Dans cette étude, son taux d'utilisation (exprimé en pourcentage) sert de métrique pour évaluer la charge imposée à chaque conteneur.

CSV — Format de fichier texte structuré dans lequel les données sont organisées en colonnes séparées par des virgules. Dans cette étude, les résultats bruts de chaque expérience (latence, taux d'échec, utilisation CPU) sont stockés dans des fichiers CSV afin de faciliter leur agrégation et leur analyse statistique ultérieure.

D

Déploiement Processus de mise en production d'une application sur une infrastructure cible, définissant comment les composants logiciels sont distribués et exécutés sur les ressources matérielles disponibles.

Docker Plateforme de conteneurisation open source permettant de créer, déployer et exécuter des applications dans des conteneurs isolés. Dans cette étude, Docker et Docker Compose sont utilisés pour orchestrer les différentes configurations expérimentales.

Docker Compose — Outil complémentaire à Docker permettant de définir et d'orchestrer plusieurs conteneurs à partir d'un fichier de configuration unique. Il est utilisé dans cette étude pour déployer et configurer automatiquement l'ensemble des services de chaque configuration expérimentale.

G

Granularité Niveau de découpage fonctionnel d'une application en services : une granularité fine correspond à des services nombreux et de petite taille, une granularité grossière à des services peu nombreux et de grande portée fonctionnelle.

H

HTTP — Protocole de communication client-serveur utilisé pour l'échange de données sur le web. Il constitue le protocole de communication inter-services retenu dans cette étude pour sa simplicité et sa compatibilité avec les environnements conteneurisés.

I

Isolation Degré de séparation physique entre les composants d'une application, matérialisé ici par le nombre de conteneurs utilisés. Une isolation forte signifie qu'un conteneur est dédié à chaque service, limitant les interférences entre composants mais augmentant la surcharge de communication.

K

Kubernetes Orchestrateur de conteneurs open source qui automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées sur un cluster de machines. Il permet notamment de répartir la charge entre les conteneurs, de redémarrer automatiquement les services défectueux et de gérer les ressources allouées à chaque composant.

L

Latence — Temps écoulé entre l'émission d'une requête par le client et la réception de la réponse du serveur, exprimé en millisecondes. Elle constitue la métrique principale pour évaluer la réactivité de chaque configuration sous différents niveaux de charge.

M

Microservice Unité atomique de l'architecture simulée, représentant une tâche de calcul simple et indépendante. Dans cette étude, chaque microservice exécute 20 000 itérations d'une division euclidienne et expose une interface HTTP pour recevoir les requêtes.

Mémoire RAM — Mémoire vive d'un ordinateur utilisée pour stocker temporairement les données en cours de traitement. Dans le contexte de cette étude, elle constitue une ressource partagée entre les conteneurs susceptible d'être saturée sous forte charge, bien qu'elle n'ait pas été retenue comme métrique principale.

Méthode QALMRI Cadre d'analyse structuré permettant de synthétiser un article scientifique en six dimensions : la Question de recherche, les Alternatives considérées, la Logique sous-jacente, la Méthode employée, les Résultats obtenus et les Inférences qui en découlent.

Monitoring Surveillance en temps réel des métriques de performance d'un système en cours d'exécution, permettant de collecter des données sur la latence, le taux d'échec et l'utilisation des ressources pour chaque configuration testée.

Multi-threadé — Qualifie un programme capable d'exécuter plusieurs threads simultanément au sein d'un même processus. Ce mode de fonctionnement est adopté pour les microservices simulés afin de se rapprocher du comportement réel d'un serveur en production face à des requêtes concurrentes.

P

Python Langage de programmation interprété utilisé dans cette étude pour simuler les microservices, générer les charges de travail et assurer le monitoring des performances en temps réel.

R

Résilience Capacité d'un système à continuer de fonctionner et à se remettre d'une défaillance partielle, mesurée dans cette étude par le taux de requêtes échouées sous différents niveaux de charge.

S

Scalabilité Capacité d'un système à maintenir des performances acceptables lorsque la charge augmente, en distribuant le traitement sur davantage de ressources sans modification de l'architecture applicative.

Service Dans le cadre de cette étude, un service désigne un regroupement logique de plusieurs microservices partageant une responsabilité fonctionnelle commune au sein d'une application distribuée simulée.

Service Weaver Framework open source développé par Google permettant de concevoir une application distribuée en Go comme un monolithe modulaire, puis de la déployer à différents niveaux de granularité sans modifier le code source. Il est utilisé dans plusieurs études de référence de cette analyse documentaire, notamment celles de Mendonça Filho & Mendonça (2024) et Medeiros Rêgo et al.[1], comme outil permettant de faire varier la granularité d'un même système de manière contrôlée et reproductible.

T

Thread — Unité d'exécution légère au sein d'un processus, permettant de traiter plusieurs tâches en parallèle sans créer de nouveaux processus. Dans cette étude, chaque microservice s'appuie sur un serveur HTTP multithreadé pour traiter les requêtes concurrentes sans blocage.

W

Windows 11 — Système d'exploitation de Microsoft utilisé comme environnement hôte dans cette étude. Docker y fonctionne via une couche de virtualisation WSL2 (Windows Subsystem for Linux), ce qui introduit une surcharge supplémentaire par rapport à un déploiement natif sous Linux et peut expliquer certaines variations de performance observées entre les configurations.

Bibliographie

- [1] C. Medeiros Rêgo, R. C. Mendonça Filho, et N. C. Mendonça, « Performance and Resilience Impact of Microservice Granularity: An Empirical Evaluation Using Service Weaver and Amazon EKS », *Int. J. Netw. Manag.*, vol. 35, n° 4, p. e70019, 2025, doi: 10.1002/nem.70019.
- [2] D. Shadija, M. Rezai, et R. Hill, « Microservices: Granularity vs. Performance », in *Companion Proceedings of the 10th International Conference on Utility and Cloud Computing*, Austin Texas USA: ACM, déc. 2017, p. 215-220. doi: 10.1145/3147234.3148093.
- [3] F. H. Vera-Rivera, C. Gaona, et H. Astudillo, « Defining and measuring microservice granularity—a literature overview », *PeerJ Comput. Sci.*, vol. 7, p. e695, sept. 2021, doi: 10.7717/peerj-cs.695.
- [4] Y. Zhao, T. D. Matteis, et J. Bogner, « How Does Microservice Granularity Impact Energy Consumption and Performance? A Controlled Experiment », in *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*, mars 2025, p. 84-95. doi: 10.1109/ICSA65012.2025.00018.
- [5] R. C. Mendonça Filho et N. C. Mendonça, « Performance Impact of Microservice Granularity Decisions: An Empirical Evaluation Using the Service Weaver Framework », in *Software Architecture*, vol. 14889, M. Galster, P. Scandurra, T. Mikkonen, P. Oliveira Antonino, E. Y. Nakagawa, et E. Navarro, Éd., in *Lecture Notes in Computer Science*, vol. 14889. , Cham: Springer Nature Switzerland, 2024, p. 191-206. doi: 10.1007/978-3-031-70797-1_13.